SafeDocs

Secure PDF Management System

CS 432 – Databases
Assignment 2: Application Development and Database Index Structure Implementation

Track 1 — Statement 6

Name	Roll Number
Sujal Patel	22110261
Siddharth Doshi	22110250
Viraj Vekaria	22110287
Nishit Mistry	22110172
Rutvi Shah	22110227

Indian Institute of Technology, Gandhinagar

Semester II (2025–2026)

Instructor: Dr. Yogesh K. Meena

GitHub Repository: https://github.com/SujalAVMP/SafeDocs

March 2026

Contents

1 Module A: Lightweight DBMS with B+ Tree Index **2**
- 1.1 Introduction . . . 2
- 1.2 Implementation Details . . . 2
  - 1.2.1 File Structure . . . 2
  - 1.2.2 B+ Tree Core (`bplustree.py`) . . . 2
  - 1.2.3 Table Abstraction (`table.py`) . . . 4
  - 1.2.4 DatabaseManager (`db_manager.py`) . . . 4
  - 1.2.5 BruteForceDB (`bruteforce.py`) . . . 4
- 1.3 Performance Analysis . . . 4
  - 1.3.1 Methodology . . . 4
  - 1.3.2 Results . . . 5
  - 1.3.3 Discussion . . . 5
- 1.4 Visualization . . . 6
- 1.5 Module A: Conclusion . . . 8
  - 1.5.1 Video Demonstration . . . 8

2 Module B: Local API Development, RBAC, and Database Optimization **9**
- 2.1 Introduction . . . 9
- 2.2 Schema Design . . . 9
  - 2.2.1 Core vs. Project-Specific Tables . . . 9
  - 2.2.2 Data Integrity . . . 10
- 2.3 API and UI Development . . . 10
  - 2.3.1 Authentication Flow . . . 10
  - 2.3.2 CRUD APIs . . . 11
  - 2.3.3 Web UI . . . 12
  - 2.3.4 Member Portfolio . . . 14
- 2.4 Security and RBAC . . . 15
  - 2.4.1 Role Definitions . . . 15
  - 2.4.2 RBAC Enforcement . . . 15
  - 2.4.3 Audit Logging . . . 15
- 2.5 SQL Indexing and Query Optimization . . . 16
  - 2.5.1 Index Selection Methodology . . . 16
  - 2.5.2 Final Index Set . . . 16
  - 2.5.3 Index Design Rationale . . . 17
- 2.6 Performance Benchmarking . . . 17
  - 2.6.1 Methodology . . . 17
  - 2.6.2 Results . . . 18
  - 2.6.3 EXPLAIN Plan Analysis . . . 18
  - 2.6.4 Discussion . . . 19
- 2.7 Module B: Conclusion . . . 20
  - 2.7.1 Video Demonstration . . . 20

3 Tools and External Sources **21**

4 Team Member Contributions **21**

1 Module A: Lightweight DBMS with B+ Tree Index

1.1 Introduction

Efficient data retrieval is a cornerstone of every database system. Real-world DBMS engines use tree-based index structures—most commonly the B+ Tree—to transform what would otherwise be linear-time searches into logarithmic-time lookups. In Module A we implement a complete B+ Tree from scratch in Python, wrap it in a relational `Table` abstraction with typed schemas and primary-key enforcement, and manage multiple databases through a `DatabaseManager` layer.

The implementation is then benchmarked against a naïve `BruteForceDB` baseline (a flat Python list with linear-scan operations) to quantify the performance advantages of structured indexing. Graphviz-based visualizations show how the tree evolves during insertions and deletions.

Problem addressed. Given a set of records identified by integer keys, we need a data structure that supports insert, delete, exact search, range query, and update in $O(\log n)$ time per operation (or $O(\log n + k)$ for range queries returning k results), while keeping the structure balanced after every mutation.

Proposed solution. A B+ Tree of configurable order, where all data resides in leaf nodes connected by a linked list, and internal nodes store only separator keys for routing.

1.2 Implementation Details

1.2.1 File Structure

The module is organized as follows:

```
Module_A/
    database/
        __init__.py          # Module exports
        bplustree.py         # B+ Tree core implementation
        bruteforce.py        # BruteForceDB baseline
        table.py             # Typed relational Table abstraction
        db_manager.py        # Multi-database DatabaseManager
    report.ipynb             # Benchmarks, visualizations, report
    requirements.txt         # graphviz, matplotlib, numpy, jupyter
```

1.2.2 B+ Tree Core (`bplustree.py`)

The B+ Tree implementation consists of two classes:

BPlusTreeNode. Each node stores a list of `keys`, a flag `is_leaf`, and either `values` (for leaf nodes) or `children` (for internal nodes). Leaf nodes additionally have a `next` pointer that links them into a doubly-connected chain for efficient range scans.

BPlusTree. The tree itself is parameterized by an `order` (minimum 3). A node can hold at most (order − 1) keys, and the minimum number of keys in a non-root node is $\lceil \text{order}/2 \rceil - 1$.

Insertion uses a proactive splitting strategy. Before descending into a child, the algorithm checks whether that child is full (has order − 1 keys). If so, the child is split *before* the recursive descent, which guarantees that we never need to propagate splits back up the call stack. The root is handled as a special case: if the root is full, a new empty root is created, the old root becomes its only child, and the old root is split.

The split operation differs between leaf and internal nodes:

- **Leaf split:** The right half of the keys and values moves to a new sibling node. The first key of the new sibling is *copied* up to the parent as a separator. The linked-list pointers are updated so the new sibling sits between the original node and whatever was previously next.
- **Internal split:** The middle key is *promoted* (pushed up) to the parent and does not remain in either child. Children pointers are divided accordingly.

Deletion handles underflow through three mechanisms:

1. **Borrow from left sibling:** If the left sibling has more than the minimum number of keys, move its last key into the underflowing node and update the parent's separator.
2. **Borrow from right sibling:** Symmetric operation on the right sibling.
3. **Merge:** If neither sibling can lend a key, merge the underflowing node with one of its siblings, consuming the parent's separator in the process. If this causes the root to become empty, the tree shrinks by one level.

After a deletion, internal separator keys that matched the deleted key are updated by finding the new minimum key of the right subtree (the leftmost leaf of that subtree).

Search traverses from the root to the appropriate leaf by comparing the target key against each node's keys to decide which child to descend into. In the leaf, a linear scan over the (small) key list finds the match.

Range query first navigates to the leaf that could contain the start key, then walks the linked list of leaves collecting all keys in the range $[\text{start}, \text{end}]$. This gives $O(\log n + k)$ performance where k is the number of matching keys.

Update navigates to the leaf and replaces the value associated with the target key. If the key does not exist, it returns `False`. Insert also handles the update case: if a key already exists, its value is overwritten in place rather than creating a duplicate.

Aggregation methods (`count`, `min_key`, `max_key`, `sum_keys`) provide $O(1)$ count (maintained via an internal `_size` counter) and $O(\log n)$ min/max by traversing to the leftmost or rightmost leaf.

Visualization uses the `graphviz` library to produce a tree diagram. Internal nodes are rendered as blue HTML tables, leaf nodes as green tables, parent-child edges as solid arrows, and leaf-to-leaf linked-list connections as dashed red arrows. This makes it easy to visually verify the tree's structure after insertions and deletions.

1.2.3 Table Abstraction (`table.py`)

The `Table` class wraps a B+ Tree to provide a relational interface:

- **Typed schema:** A dictionary mapping column names to Python types (e.g., {`"student_id": int, "name": str`}). On insert and update, every value is validated against the schema. Values that are not already the correct type are coerced if possible; otherwise the operation fails with a descriptive error message.
- **Primary key enforcement:** One column is designated as the `search_key`. The B+ Tree uses this column's value as the key. Duplicate primary keys are rejected.
- **CRUD operations:** `insert`, `get`, `get_all`, `update`, `delete`—all returning `(success, result/message)` tuples for consistent error handling.
- **Constraint-based search:** A linear scan over all records that match a set of {`column: value`} constraints.
- **Range query:** Supports two calling conventions—a two-argument form `(start, end)` that uses the B+ Tree's native range scan on the search key, and a three-argument form `(field, start, end)` that falls back to a linear scan for non-indexed columns.
- **Partial update:** The `update` method accepts a subset of columns; unchanged columns retain their original values.

1.2.4 DatabaseManager (`db_manager.py`)

The `DatabaseManager` holds a dictionary of named databases, each of which is itself a dictionary of named `Table` instances. It provides DDL-like methods: `create_database`, `delete_database`, `create_table`, `delete_table`, `list_databases`, `list_tables`, and `get_table`.

This mirrors the multi-database structure of a real DBMS: you can create separate databases for different purposes (e.g., `"safedocs"`, `"analytics"`) and manage their tables independently.

1.2.5 BruteForceDB (`bruteforce.py`)

The baseline stores `(key, value)` tuples in a flat Python list. Every operation—insert, search, delete, range query—is $O(n)$ because it scans the entire list. Insert additionally performs a duplicate check before appending. Range query and `get_all` sort the results before returning them, adding an $O(n \log n)$ overhead.

This class exists solely as a performance comparison target. It implements the same interface as the B+ Tree, which allows the benchmark code to treat both data structures uniformly.

1.3 Performance Analysis

1.3.1 Methodology

The `PerformanceAnalyzer` class in the Jupyter notebook runs benchmarks across 12 dataset sizes ranging from 100 to 20,000 keys. For each size:

1. A set of random integer keys is generated.
2. All keys are inserted into both a fresh B+ Tree and a fresh BruteForceDB.

3. A random subset of 100 keys is searched, deleted, and used for range queries.
4. 200 random operations (mix of insert, search, delete) are executed.
5. Peak memory usage is measured using `sys.getsizeof` recursively on all internal structures.

Each measurement is timed using Python's `time.perf_counter()` for microsecond-level precision. Results are collected into a structured dictionary and plotted using Matplotlib.

1.3.2 Results

Table 1 shows the summary at the largest dataset size (20,000 keys).

Table 1: B+ Tree vs. BruteForceDB at 20,000 keys

Metric	**B+ Tree**	**BruteForce**	**Speedup**
Insertion (20K keys)	0.123 s	4.666 s	38.1×
Search (100 keys)	0.000 31 s	0.011 s	36.1×
Deletion (100 keys)	0.000 55 s	0.026 s	47.0×
Range Query (100 keys)	0.000 82 s	0.041 s	50.5×
Random Ops (200)	0.000 77 s	0.029 s	37.2×
Peak Memory	3,572 KB	1,155 KB	0.32×

1.3.3 Discussion

Insertion. The B+ Tree's proactive splitting strategy keeps insertion at $O(\log n)$ per key, whereas BruteForceDB's duplicate-check scan makes each insertion $O(n)$, leading to $O(n^2)$ total insertion time. The 38× speedup at 20K keys reflects this quadratic vs. $O(n \log n)$ gap.

Search. Both structures search for the same random set of 100 keys. The B+ Tree reaches the correct leaf in $O(\log n)$ steps, while the brute-force approach scans the entire list, giving a 36× speedup.

Deletion. The B+ Tree locates the key in $O(\log n)$ and rebalances in amortized constant time per level. The brute-force approach must scan to find the key and then shift elements after removal. The 47× speedup is consistent with the search speedup plus the additional cost of list compaction in the brute-force case.

Range queries. This is where the B+ Tree shows its largest advantage (50.5×). The leaf linked list allows the tree to scan only the matching keys in $O(\log n + k)$ time, while the brute-force approach must scan all n keys and then sort the matches. As the dataset grows, this gap widens significantly.

Memory. The B+ Tree uses approximately 3× more memory than the brute-force structure. This overhead comes from node objects, child pointer arrays, and the linked-list pointers in leaf nodes. This is the classic time–space tradeoff: the tree trades higher memory usage for dramatically lower operation times.

The Matplotlib plots in Figure 1 show that all time metrics grow linearly for the B+ Tree (reflecting the $O(\log n)$ per-operation cost summed over the operations) while the brute-force metrics show clear quadratic or super-linear growth.

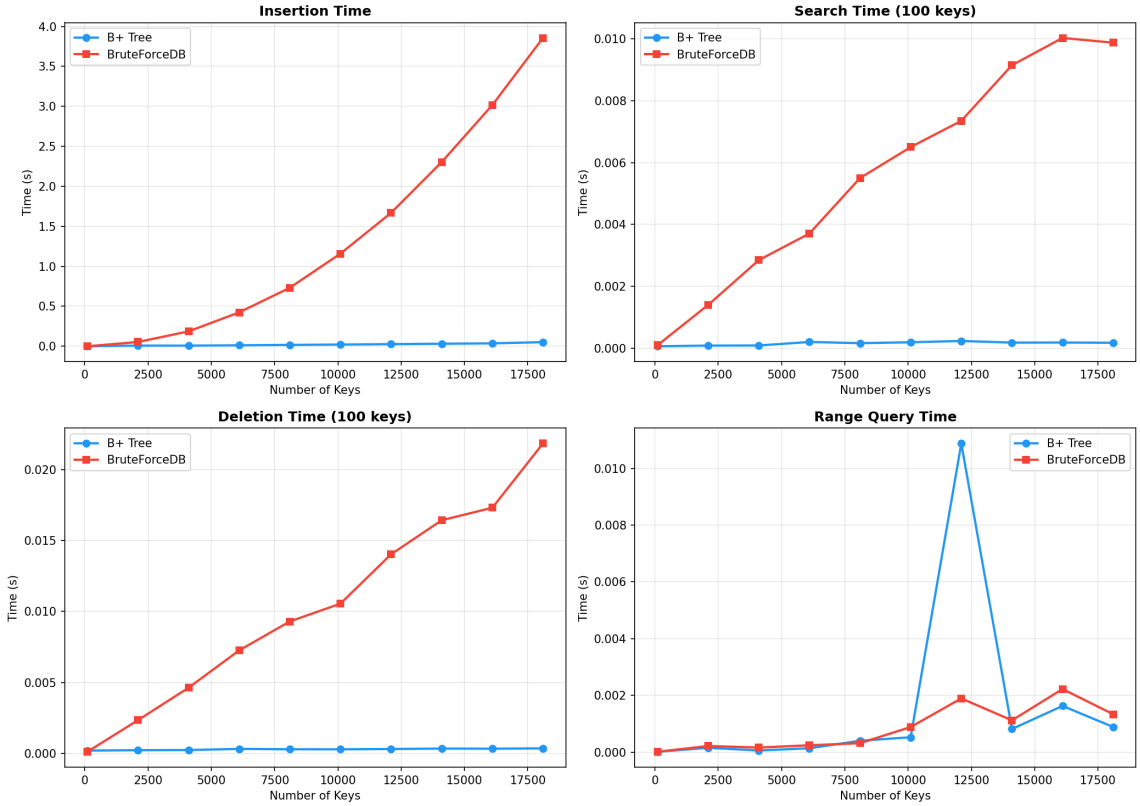


Figure 1: B+ Tree vs. BruteForceDB performance across dataset sizes (100–20,000 keys). The B+ Tree (blue) scales near-linearly while the brute-force approach (red) shows quadratic growth.

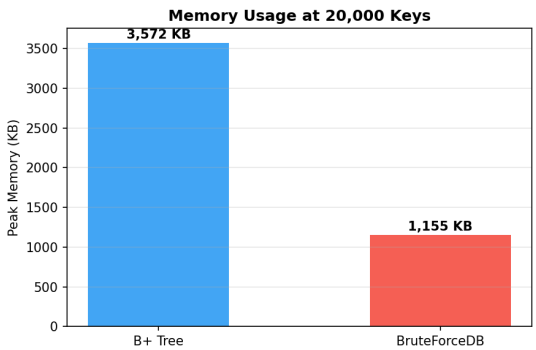


Figure 2: Peak memory usage at 20,000 keys. The B+ Tree uses ∼3× more memory due to node objects and pointer overhead—the classic time–space tradeoff.

1.4 Visualization

The Graphviz visualizations in the notebook demonstrate several key properties of the B+ Tree:

- **Tree structure:** Internal nodes (blue) contain only separator keys; leaf nodes (green) hold the actual key–value data. The tree is always balanced—all leaves are at the same depth.

- **Node relationships:** Parent–child edges are solid black arrows. Each internal node's separators divide the key space among its children.
- **Leaf linkage:** Red dashed arrows connect adjacent leaves from left to right, forming the linked list used for range scans.
- **Splitting:** A step-by-step visualization inserts keys one at a time into an order-3 tree, showing each split as it happens. When a leaf overflows, the middle key is copied up; when an internal node overflows, the middle key is promoted.
- **Deletion and rebalancing:** A before/after visualization deletes several keys from a populated tree, showing how nodes merge and redistribute to maintain the minimum-key invariant.

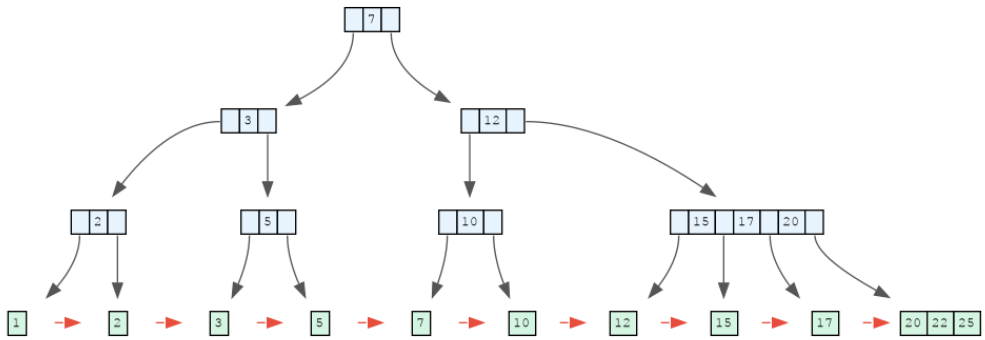


Figure 3: B+ Tree structure for a SafeDocs document index (order 4, 12 keys). Blue nodes are internal separators; green nodes are leaves holding data. Red dashed arrows show the leaf linked list.

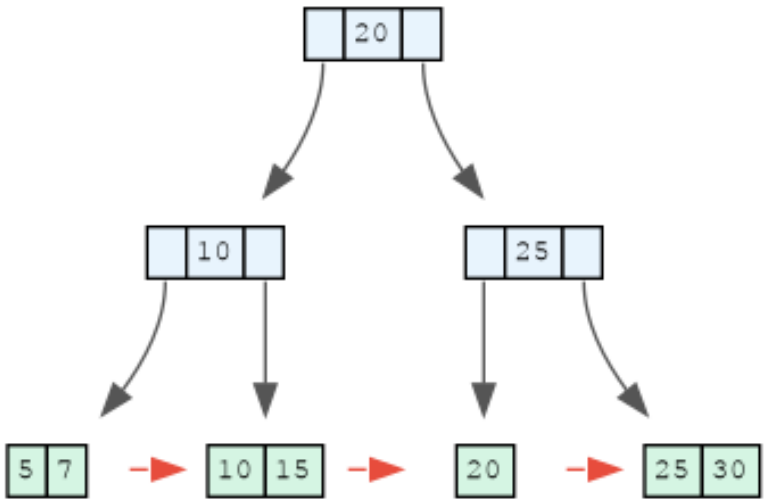
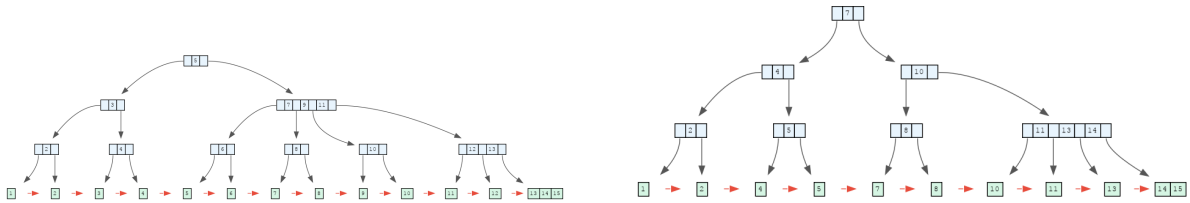


Figure 4: B+ Tree after step-by-step insertion of keys [10, 20, 5, 15, 25, 30, 7] into an order-3 tree. Two splits have occurred, producing a two-level tree with the root separator correctly routing queries.



(a) Before deletion (keys 1–15) (b) After deleting keys 3, 6, 9, 12

Figure 5: B+ Tree rebalancing after deletion. Nodes merge and redistribute to maintain the minimum-key invariant while keeping all leaves at the same depth.

1.5 Module A: Conclusion

The B+ Tree implementation satisfies all required operations (insert, delete, search, range query, update, value storage) with correct node splitting, merging, and leaf-node linkage. The performance benchmarks confirm the expected $O(\log n)$ vs. $O(n)$ complexity difference, with the B+ Tree outperforming the brute-force baseline by 36–50× across all timed metrics at 20,000 keys. The only tradeoff is a 3× increase in memory usage, which is acceptable given the dramatic time savings.

These results demonstrate why the B+ Tree is the dominant index structure in production database systems: it provides fast point lookups, efficient range scans via the leaf linked list, and predictable performance even as the dataset grows.

1.5.1 Video Demonstration

Module A Video: https://youtu.be/redhcqXd5tg

2 Module B: Local API Development, RBAC, and Database Optimization

2.1 Introduction

Module B transforms the SafeDocs database schema designed in Assignment 1 into a fully functional local web application. The implementation uses Flask (Python) as the web framework, MySQL as the database, and PyJWT for session-based authentication. The system provides:

- A web UI with login, dashboard, member management, document management, and portfolio views.
- RESTful CRUD APIs for Members, Documents, Departments, Folders, and Security Logs.
- JWT-based session validation on every protected route.
- Role-Based Access Control (RBAC) with five roles: Admin, Manager, Editor, Viewer, and Auditor.
- Dual audit logging to both a `SecurityLog` database table and an `audit.log` file.
- Document-centric activity logging to the `AccessLog` table for dashboard and portfolio views.
- SQL indexing aligned with actual application queries, with before/after benchmarking.

2.2 Schema Design

2.2.1 Core vs. Project-Specific Tables

The database uses 14 tables in total. The original 12 project-specific tables from Assignment 1 (Department, Role, Member, Folder, Document, DocumentVersion, Tag, DocumentTag, Permission, AccessLog, SharedLink, Notification) handle the document management domain. Module B adds two core tables for authentication and security auditing:

- **UserLogin** – Stores authentication credentials (username and password hash) separately from the `Member` table. This separation ensures that login credentials are never duplicated inside project-specific tables. Each `UserLogin` row references a `Member` via a foreign key, and the `Username` column has a `UNIQUE` constraint.
- **SecurityLog** – Records every API action (logins, CRUD operations, denied access attempts) with the member ID, action name, affected table, record ID, IP address, session validity flag, and a free-text details field.

Both tables are created by `Module_B/sql/setup.sql`, which is idempotent (`CREATE TABLE IF NOT EXISTS`) and can be re-run safely.

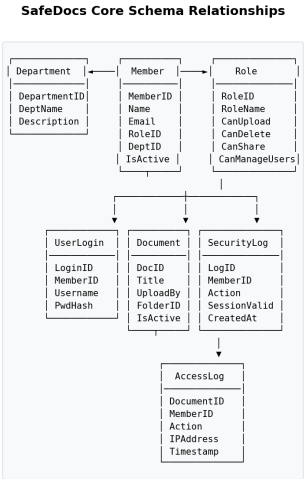


Figure 6: Core schema relationships in Module B. `Member` is the central entity, linked to `Department` and `Role` (from Assignment 1) and to the new `UserLogin` and `SecurityLog` tables. Document CRUD activity flows into `AccessLog`.

2.2.2 Data Integrity

- **Member ↔ UserLogin:** When a new member is created through the API, a corresponding `UserLogin` entry is created in the same transaction. When a member is deleted, the `UserLogin` entry is removed first (the `ON DELETE CASCADE` constraint also handles this automatically).
- **Soft deletes:** Documents are never physically deleted. The `DELETE` API endpoint sets `IsActive = FALSE` on the document record, preserving the full audit trail while hiding the document from active listings.
- **Foreign keys:** All relationships use appropriate `ON UPDATE CASCADE` and either `ON DELETE RESTRICT` (to prevent orphaning critical references) or `ON DELETE SET NULL` (for non-critical references like folder parents and security log members).

2.3 API and UI Development

2.3.1 Authentication Flow

Password hashing. Passwords are hashed using SHA-256 (`hashlib.sha256`), matching MySQL's `SHA2(x, 256)` function. This means the application-side hash and the database-side hash are identical, and passwords seeded through SQL (using `SHA2('password', 256)`) work with the Flask login endpoint without conversion.

Login. The `POST /login` endpoint accepts both JSON (`user/password`) and form-encoded data (`username/password`). It looks up the username in `UserLogin`, joins with `Member` and `Role` to retrieve the member's identity and role, verifies the password hash, and issues a JWT session token containing the member ID, role ID, role name, member name, issued-at timestamp, and expiry (24 hours).

For browser requests, the token is set as an `httponly` cookie with `SameSite=Lax`. For API requests, the token is returned in the JSON response body.

Session validation. Every protected route uses the `@login_required` decorator, which:

1. Extracts the JWT from the cookie or the `Authorization: Bearer` header.
2. Calls `verify_session_token_detailed()`, which returns a `(payload, status)` tuple. The status is one of `"ok"`, `"expired"`, or `"invalid"`.
3. If the token is invalid, API routes return the appropriate JSON error (401); UI routes redirect to the login page.
4. If the token is valid, the user's identity (`member_id`, `role_id`, `role_name`, `member_name`) is attached to Flask's request-scoped `g` object for use by the route handler.

isAuth endpoint. The `GET /isAuth` endpoint lets clients check whether their session is still valid. It returns the username, role, and expiry time on success, or a specific error (`"No session found"`, `"Session expired"`, `"Invalid session token"`) on failure.

2.3.2 CRUD APIs

All CRUD endpoints follow a consistent pattern: validate the session, check role permissions, execute the database operation, log the action, and return a JSON response with an appropriate HTTP status code.

Members API.

- `GET /api/members` – Admins see all members (with optional `?name=` filter); regular users see only their own record.
- `GET /api/members/<id>` – Returns a single member. Admins, Managers, and Auditors can view any member; others can only view themselves.
- `POST /api/members` – Admin only. Creates a new member and a corresponding `UserLogin` entry. Required fields: Name, Email, ContactNumber, DepartmentID, RoleID.
- `PUT /api/members/<id>` – Admins can update any member; others can only update themselves. Admins can additionally change DepartmentID and RoleID.
- `DELETE /api/members/<id>` – Admin only. Deletes the member and their `UserLogin` entry.

Documents API.

- `GET /api/documents` – Lists active documents with optional `?title=` filter. Joins with Member and Folder to show uploader name and folder name.
- `GET /api/documents/<id>` – Returns a single active document.
- `POST /api/documents` – Requires the `CanUpload` role capability. Creates a new document and logs an `UPLOAD` activity to `AccessLog`.
- `PUT /api/documents/<id>` – Requires `CanUpload` or document ownership. Updates allowed fields (Title, Description, IsConfidential, FolderID) and logs an `EDIT` activity.
- `DELETE /api/documents/<id>` – Requires the `CanDelete` role capability. Performs a soft delete (`IsActive = FALSE`) and logs a `DELETE` activity.

Supporting APIs.

- `GET /api/departments` – Lists all departments.

- `GET /api/folders` – Lists all active folders with creator names.
- `GET /api/security-logs` – Admin and Auditor only. Lists the 100 most recent security log entries, with optional `?session_valid=true|false` filter.

2.3.3 Web UI

The web UI uses Bootstrap 5 with Jinja2 templates. All pages extend a common `base.html` that provides the navigation bar, session info display, and flash message support.

- **Login page** (`login.html`) – Username/password form.
- **Dashboard** (`dashboard.html`) – Shows system statistics (total members, documents, folders, departments) and the 10 most recent document activities from the `AccessLog` table.
- **Members page** (`members.html`) – Lists all members. Admins see edit/delete actions.
- **Member edit page** (`member_edit.html`) – Form to edit member details.
- **Documents page** (`documents.html`) – Lists active documents. Users with upload permission see a "New Document" button; users with delete permission see delete actions.
- **Document edit page** (`document_edit.html`) – Form for creating or editing documents.
- **Portfolio page** (`portfolio.html`) – Displays a member's profile, document count, and the 10 most recent document activities. Non-admin users can only view their own portfolio.

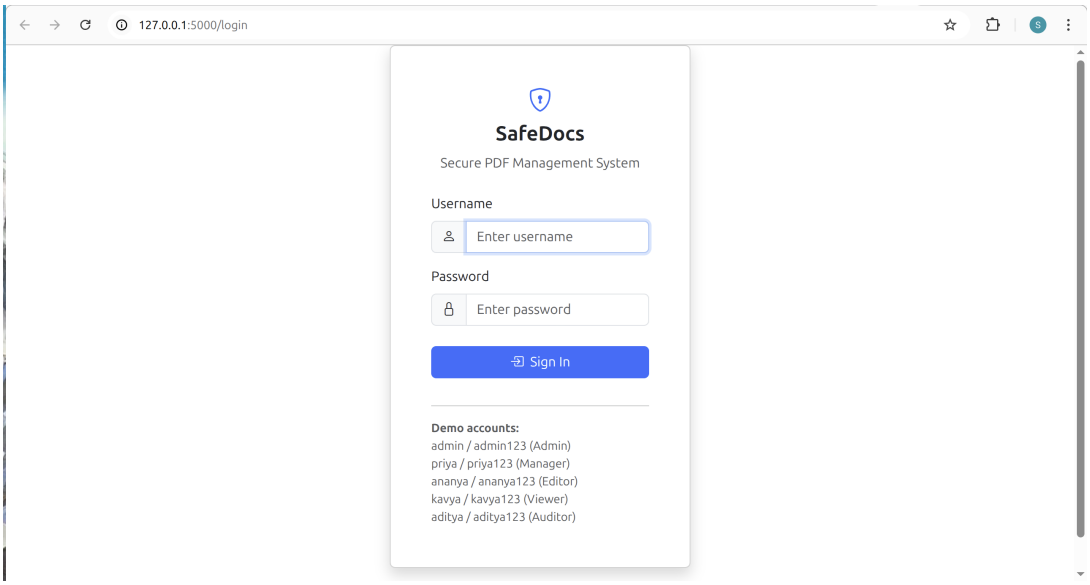


Figure 7: SafeDocs login page with username/password form.

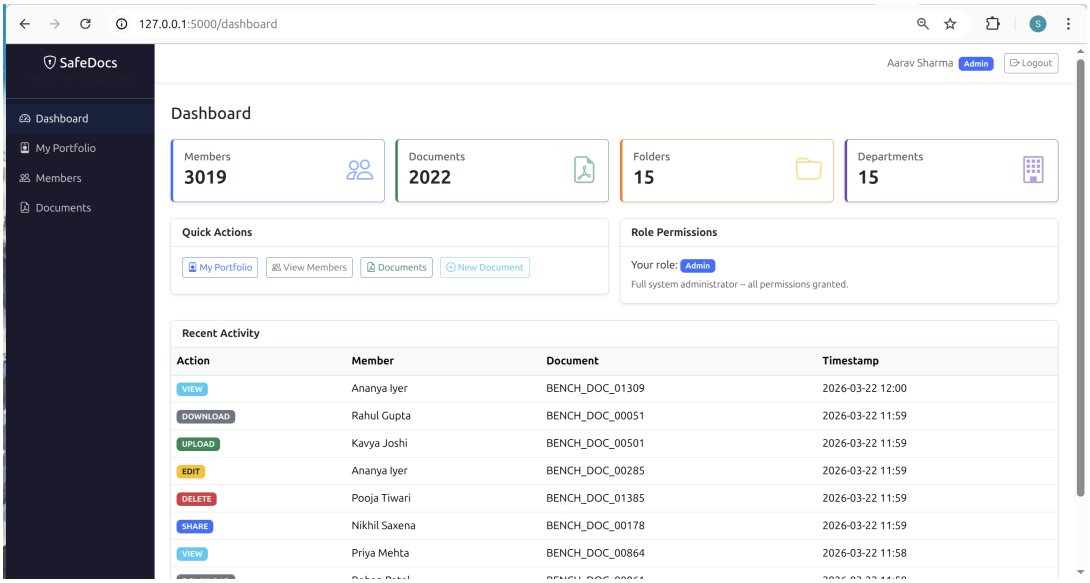


Figure 8: Admin dashboard showing system statistics and recent document activity from the AccessLog table.

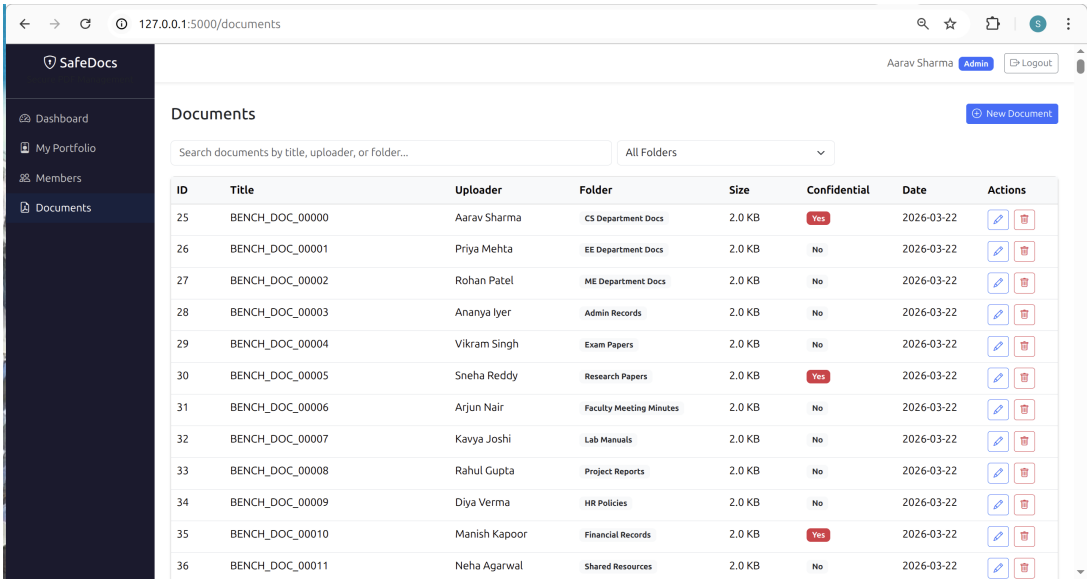


Figure 9: Document listing page. Admin users see create/edit/delete actions; restricted roles see only the listing.

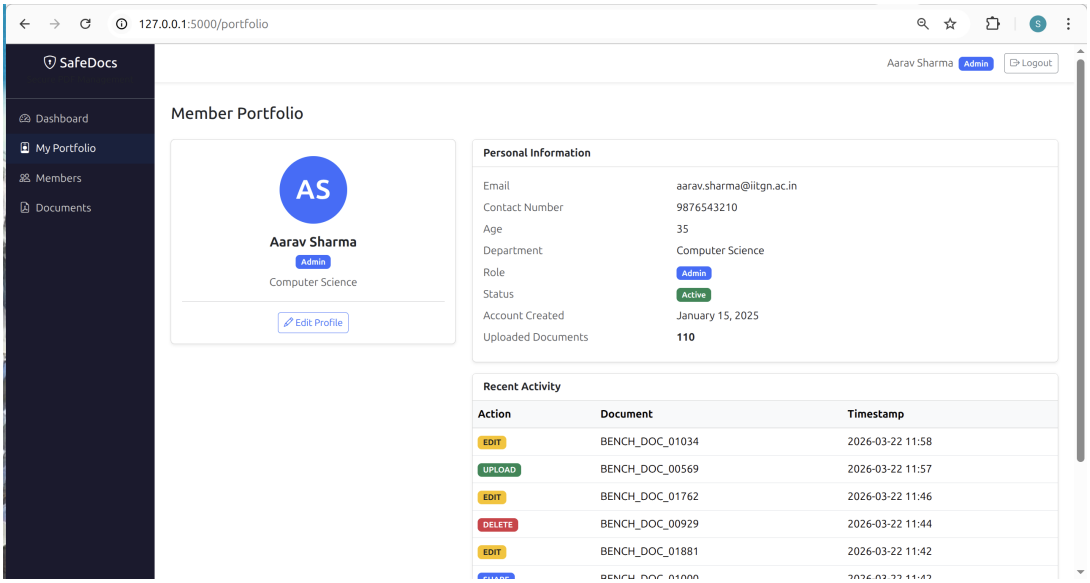


Figure 10: Member portfolio showing profile details, document count, and recent activity timeline.

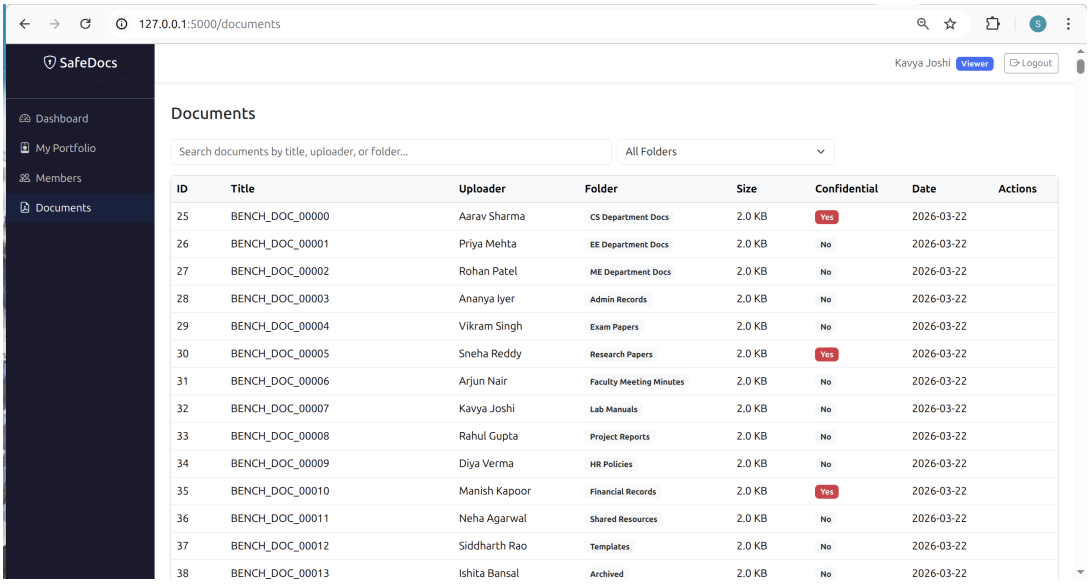


Figure 11: Documents page as a Viewer role—no create or delete actions are available, demonstrating RBAC enforcement at the UI level.

2.3.4 Member Portfolio

The portfolio feature is implemented at `/portfolio` and `/portfolio/<member_id>`. It is protected by `@login_required` and enforces the following access rules:

- If no `member_id` is provided, the logged-in user's own portfolio is shown.
- Admin users (RoleID = 1) can view any member's portfolio.
- Non-admin users attempting to view another member's portfolio are redirected to their own portfolio with a warning flash message.

The portfolio page queries `Member` (profile), `Document` (document count), and a subquery

on `AccessLog` (recent activity). The subquery pattern `SELECT ... FROM (SELECT ... FROM AccessLog WHERE MemberID = ? ORDER BY AccessTimestamp DESC LIMIT 10) AS recent JOIN Document ...` is specifically designed to benefit from the composite index `idx_accesslog_member_timestamp`.

2.4 Security and RBAC

2.4.1 Role Definitions

The `Role` table defines five roles with boolean capability flags:

Table 2: Role capability matrix

Role	CanUpload	CanDelete	CanShare	CanManageUsers
Admin	✓	✓	✓	✓
Manager	✓	✓	✓	–
Editor	✓	–	✓	–
Viewer	–	–	–	–
Auditor	–	–	–	–

2.4.2 RBAC Enforcement

Access control is enforced at two levels:

1. **Decorator level:** The `@admin_required` decorator wraps `@login_required` and additionally checks that `g.role_id == 1`. It is applied to administrative endpoints (member creation, member deletion).
2. **Route level:** Individual routes query the `Role` table (via `_get_role_caps()`) to check specific capabilities. For example, document creation checks `CanUpload`, document deletion checks `CanDelete`, and security log access checks whether the role is Admin or Auditor.

When a user lacks the required capability, the system:

- Returns `403 Forbidden` with a descriptive error message for API routes.
- Logs the denied attempt to both `SecurityLog` and `audit.log` (e.g., `API_CREATE_DOCUMENT_DENIED`
- For UI routes, redirects with a flash warning message.

2.4.3 Audit Logging

The system maintains two separate logging streams:

SecurityLog table + audit.log file. The `log_action()` function writes every significant event to both destinations atomically. Each entry records:

- `MemberID` – who performed the action (NULL for failed logins with unknown users)
- `Action` – what happened (e.g., `LOGIN_SUCCESS`, `API_CREATE_DOCUMENT`, `API_DELETE_DOCUMENT_DEN`
- `TableName` and `RecordID` – which database record was affected
- `IPAddress` – the client's IP address

- `SessionValid` – whether the request had a valid session (FALSE for failed logins and expired sessions)
- `Details` – free-text context (e.g., username for login attempts, field list for updates)

This dual logging ensures that even if the database is compromised, the file-based log retains an independent audit trail.

AccessLog table. The `log_document_activity()` function writes document-centric events (UPLOAD, EDIT, DELETE) to the `AccessLog` table. This feeds the dashboard's "recent activity" widget and the portfolio's activity timeline. The `AccessLog` uses the same ENUM values defined in the schema (`VIEW`, `DOWNLOAD`, `UPLOAD`, `EDIT`, `DELETE`, `SHARE`, `PERMISSION_CHANGE`).

Detecting unauthorized modifications. Any direct database modification made without going through the session-validated APIs will have no corresponding `SecurityLog` entry. By comparing the database state against the security log timeline, unauthorized changes can be identified. For example, if a document's `Title` changed at time T but there is no `API_UPDATE_DOCUMENT` log entry near time T, the change was made directly on the database.

2.5 SQL Indexing and Query Optimization

2.5.1 Index Selection Methodology

Rather than creating indexes based on the schema in isolation, we analyzed the actual SQL queries executed by the Flask application's routes. For each route, we identified the `WHERE`, `JOIN`, and `ORDER BY` clauses and selected indexes that would provide the most benefit.

2.5.2 Final Index Set

Table 3: Indexes created in `setup.sql`

Index Name	Columns	Supporting Route
`idx_document_title`	`Document(Title)`	`/api/documents?title=...`
`idx_member_name`	`Member(Name)`	`/api/members?name=...`
`idx_accesslog_member_timestamp`	`AccessLog(MemberID, AccessTimestamp)`	`/portfolio/<id>`
`idx_folder_active_name`	`Folder(IsActive, FolderName)`	`/api/folders`
`idx_securitylog_session_created`	`SecurityLog(SessionValid, CreatedAt)`	`/api/security-logs ?session_valid=...`

`UserLogin.Username` already has a `UNIQUE` constraint, which MySQL automatically indexes, so the login lookup (`WHERE Username = ?`) is already optimized without an additional index.

2.5.3 Index Design Rationale

- `idx_document_title`: The document listing API supports an exact title filter. Without an index, MySQL scans all rows of the `Document` table. With the index, it performs a direct key lookup, reducing scanned rows from thousands to one.
- `idx_member_name`: Similar to the document title index—the admin member filter uses `WHERE m.Name = ?`, which benefits from a direct index lookup.
- `idx_accesslog_member_timestamp`: The portfolio activity query filters by `MemberID` and orders by `AccessTimestamp DESC`. A composite index on both columns allows MySQL to satisfy the `WHERE` and `ORDER BY` in a single index range scan, avoiding a filesort. The column order matters: `MemberID` first (equality predicate) then `AccessTimestamp` (range/ordering).
- `idx_folder_active_name`: The folder listing filters by `IsActive = TRUE` and orders by `FolderName`. The composite index supports both the filter and the sort. However, since the Folder table is small (typically fewer than 20 rows), the performance impact is minimal.
- `idx_securitylog_session_created`: The security log API filters by `SessionValid` and orders by `CreatedAt DESC`. The composite index allows MySQL to scan only the matching `SessionValid` partition in reverse `CreatedAt` order. However, when a large fraction of rows have the same `SessionValid` value, the selectivity is limited and the speedup is modest.

2.6 Performance Benchmarking

2.6.1 Methodology

The benchmarking process is fully automated in `Module_B/report.ipynb`:

1. **Synthetic data generation:** The notebook inserts 3,000 benchmark members, 2,000 benchmark documents, 15,000 access log entries, and 8,000 security log entries to ensure the tables are large enough for indexes to make a measurable difference.
2. **Before-indexes measurement:** All five candidate indexes are dropped. Four SQL queries (matching the Flask routes) are each run 25 times, and the median execution time is recorded. Three API endpoints are similarly timed using Flask's `test_client()`.
3. **After-indexes measurement:** The five indexes are recreated. The same queries and API endpoints are re-measured.
4. **EXPLAIN comparison:** `EXPLAIN` output is captured before and after indexing to show how MySQL's query planner changes its access strategy.

SQL timings use `time.perf_counter()` around individual query executions. API timings measure the full round-trip through Flask (routing, decorator checks, database query, JSON serialization) using the same `perf_counter` approach.

2.6.2 Results

Table 4: SQL query execution times (median, microseconds)

Query	Before (μs)	After (μs)	Speedup
Document Filter	1,705	373	4.57×
Member Filter	2,084	309	6.75×
Portfolio Activity	407	630	0.65×
Security Logs	2,546	2,551	1.00×

Table 5: API response times (median, microseconds)

Endpoint	Before (μs)	After (μs)	Speedup
Document Filter API	2,141	625	3.43×
Member Filter API	4,266	732	5.83×
Security Logs API	3,266	1,953	1.67×

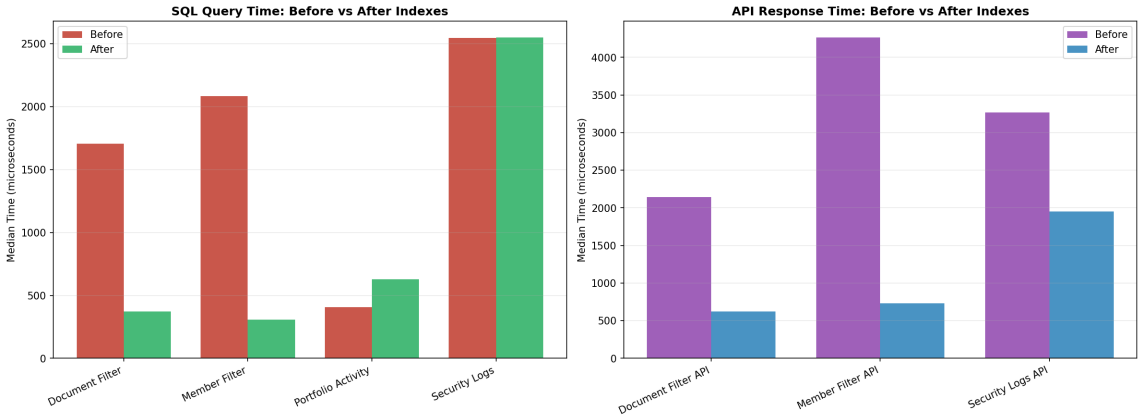


Figure 12: Module B benchmark results. Left: SQL query execution times before and after indexing. Right: API response times before and after indexing. Document and member filters show the strongest improvement (4–7×).

2.6.3 EXPLAIN Plan Analysis

The `EXPLAIN` output shows concrete changes in MySQL's access plan:

Document Filter.

- **Before:** MySQL starts with a full scan of the `Folder` table (`type=ALL`, 15 rows), then does an index lookup on `Document` using `idx_document_folder` (67 rows per folder), then a primary key lookup on `Member`.
- **After:** MySQL starts with the `Document` table using `idx_document_title` (`type=ref`, 1 row), then does primary key lookups on both `Member` and `Folder`. The join order reverses because the optimizer recognizes that the new index provides much higher selectivity.

Member Filter.

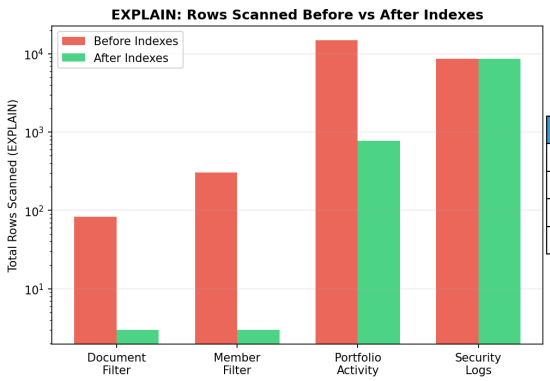
- **Before:** MySQL starts with an index scan of the `Role` table (5 rows), then scans `Member` using `idx_member_role` (301 rows per role), then looks up `Department`. It must scan ∼1,505 member rows to find the one matching name.
- **After:** MySQL starts with `Member` using `idx_member_name` (`type=ref`, 1 row), then primary key lookups on `Department` and `Role`. Total rows scanned drops from ∼1,500 to 3.

Portfolio Activity.

- **Before:** The inner `AccessLog` query uses `idx_accesslog_timestamp` (a single-column index on timestamp), scanning 14,808 rows.
- **After:** The inner query uses `idx_accesslog_member_timestamp` (the new composite index), scanning only 769 rows for the specific member. Despite this improvement in the plan, the wall-clock time does not decrease because the LIMIT 10 on the subquery already constrains the result size, and the overhead of index maintenance slightly offsets the gain on this small result set.

Security Logs.

- **Before:** Full table scan (`type=ALL`, 8,593 rows).
- **After:** Range scan using `idx_securitylog_session_created` (`type=range`, 8,584 rows). The plan quality improves, but because the `SessionValid = TRUE` predicate matches nearly all rows in the benchmark dataset, the selectivity is low and the SQL-level speedup is negligible. The API-level speedup (1.67×) is slightly better because the index also helps with the `ORDER BY CreatedAt DESC` clause, avoiding a filesort.



EXPLAIN: Access Type Changes

Query	Before (access type)	After (access type)
Document Filter	ALL (Folder)	ref (Document)
Member Filter	index (Role)	ref (Member)
Portfolio Activity	index (AccessLog)	ref (AccessLog)
Security Logs	ALL (SecurityLog)	range (SecurityLog)

Figure 13: Left: Total rows scanned by MySQL (log scale) before and after indexing—document and member filters drop from thousands to single digits. Right: Access type changes showing how the optimizer switches from full scans and index scans to targeted ref/range lookups.

2.6.4 Discussion

The benchmarks confirm that indexing provides the strongest benefit when the predicate is highly selective (exact-match on a unique or near-unique column). The document title

and member name indexes produced 4.6× and 6.8× speedups respectively because each filter matches exactly one row out of thousands.

Composite indexes help with combined filter+sort patterns (portfolio activity) but show smaller wall-clock improvements when the query already returns a small result set (LIMIT 10).

Low-cardinality filters (security log `SessionValid`, which is essentially a boolean) gain little from indexing because the index does not meaningfully reduce the number of rows scanned. In a production system with a more balanced distribution of valid/invalid sessions, this index would be more effective.

2.7 Module B: Conclusion

Module B delivers a complete local web application that satisfies all assignment requirements:

1. **Local Database & Secure API:** The SafeDocs schema is fully operational with 14 tables, CRUD APIs for all core entities, session validation on every protected route, and a member portfolio feature restricted to authorized users.
2. **Security & RBAC:** Five roles with distinct capability flags enforce access control at both the decorator and route levels. Failed access attempts are logged. Member deletion correctly removes the associated `UserLogin` entry to maintain data integrity.
3. **Database Optimization:** Five indexes were created based on actual application query patterns. Before/after benchmarking shows speedups of up to 6.75× on targeted queries.
4. **Optimization Report:** This report documents the schema design, security model, indexing rationale, and quantitative benchmarking results with EXPLAIN plan comparisons.

2.7.1 Video Demonstration

Module B Video: https://youtu.be/ETjbWK0PtVo

3 Tools and External Sources

As required by the submission guidelines, we disclose all tools and external resources used:

- **Programming language:** Python 3.10+
- **Web framework:** Flask 3.x
- **Database:** MySQL 8.0
- **Authentication:** PyJWT (JWT token generation and verification)
- **Database driver:** PyMySQL
- **Visualization:** Graphviz (B+ Tree diagrams), Matplotlib (benchmark plots)
- **Frontend:** Bootstrap 5 (CSS framework), Jinja2 (templating)
- **AI assistance:** Claude (Anthropic) – used for code review, debugging assistance, and report drafting
- **Reference material:** B+ Tree playlist referenced in the assignment specification; MySQL documentation for EXPLAIN output interpretation

4 Team Member Contributions

Name	Contribution
Sujal Patel (22110261)	Module B: Flask app, APIs, RBAC, video production
Siddharth Doshi (22110250)	Module A: B+ Tree implementation and benchmarking
Viraj Vekaria (22110287)	Module A: Visualization and performance analysis
Nishit Mistry (22110172)	Module B: SQL setup, indexing strategy, benchmarking
Rutvi Shah (22110227)	Schema design, testing, report